

Languages, automata, and the Chomsky hierarchy

Computational modeling of language
Spring 2026

Bureaucracy

Bureaucracy

- **Waitlist:** Working on it.

Bureaucracy

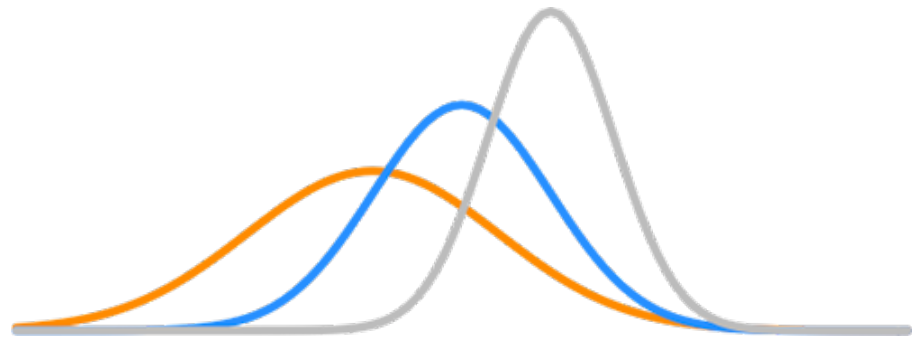
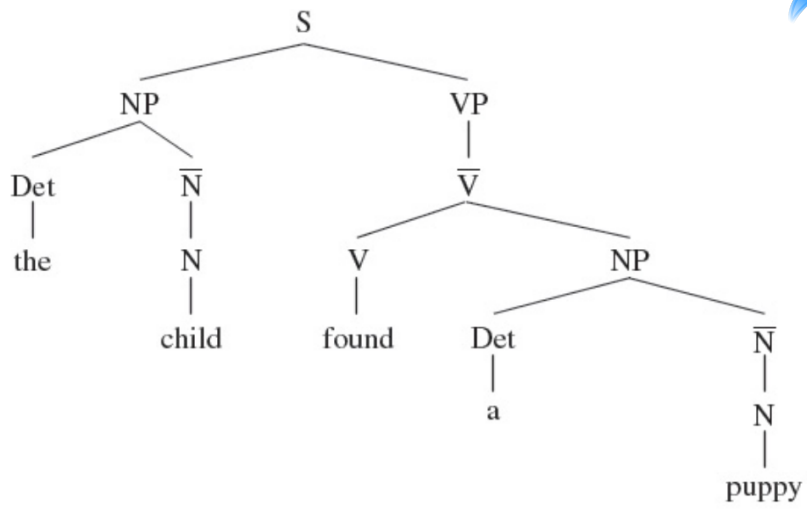
- **Waitlist:** Working on it.
- **In case of illness:** Please stay home and get better - we will accommodate!

Bureaucracy

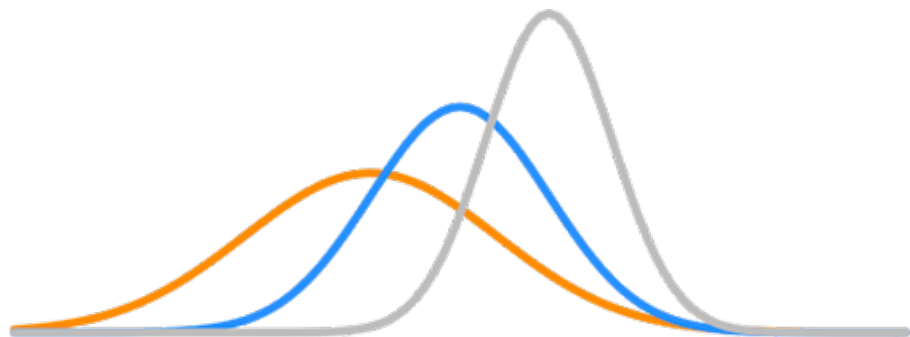
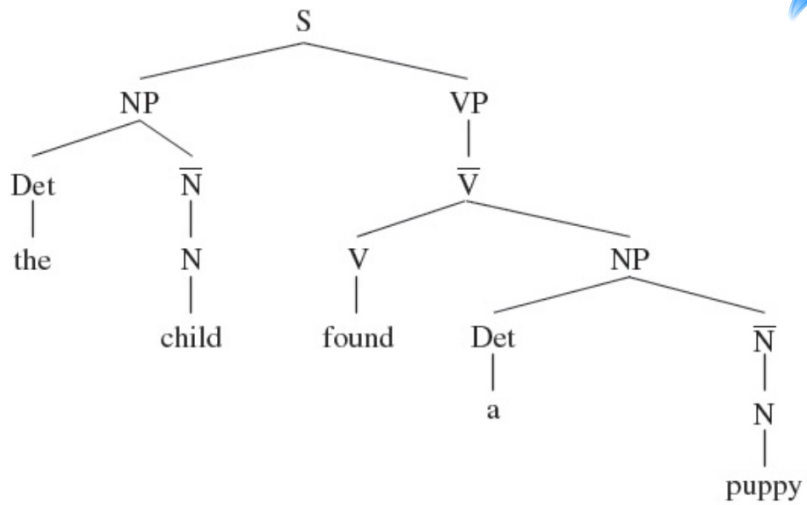
- **Waitlist:** Working on it.
- **In case of illness:** Please stay home and get better - we will accommodate!
- **Announcement:** By Nicki Young of SLUGS.

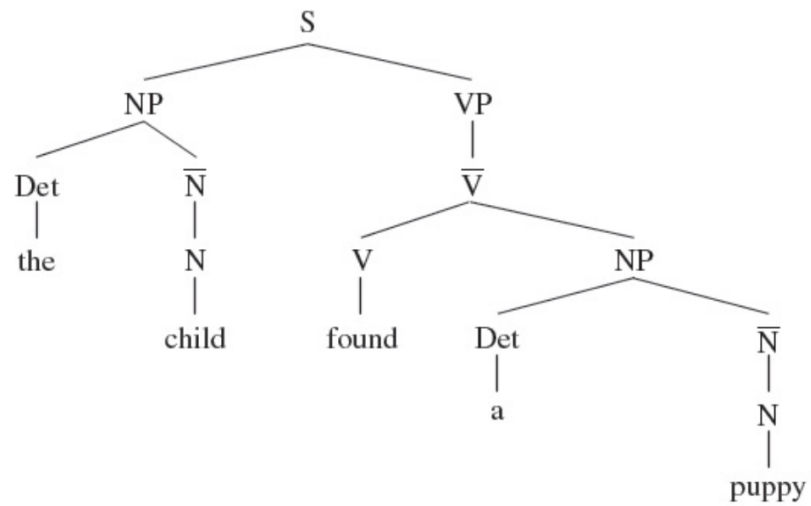


Linguistics and
computation
share **deep roots**.



What **comments** or **questions**
do you have from last time?



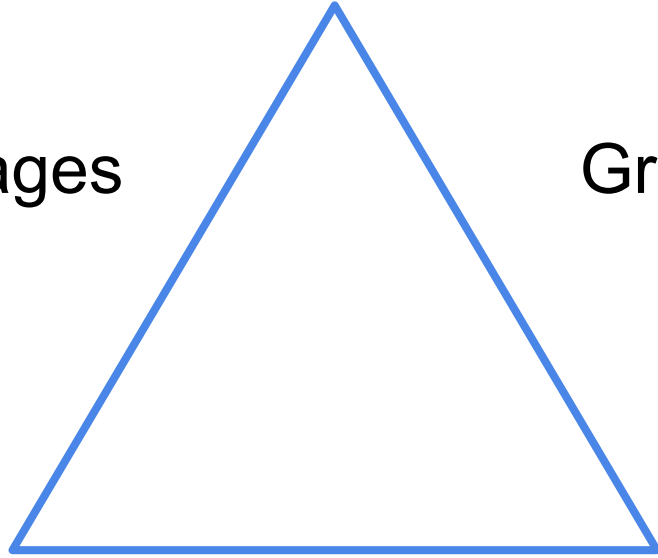


Overview of this presentation

- Languages, machines, and grammars
- Finite state machines
- The Chomsky hierarchy

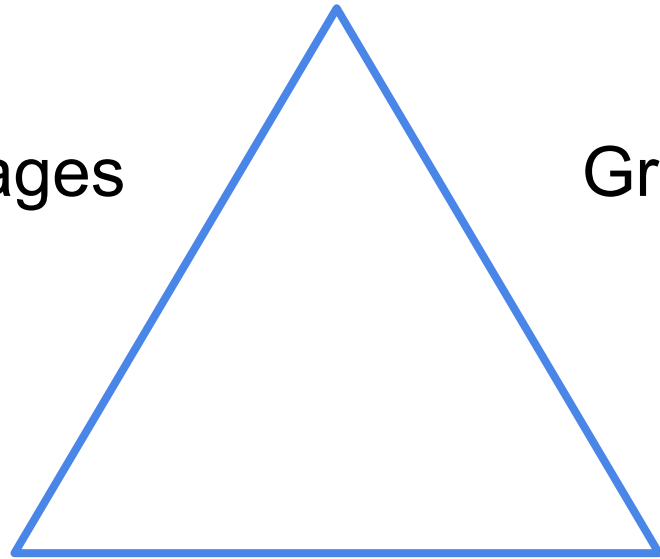
Languages

Grammars



Languages

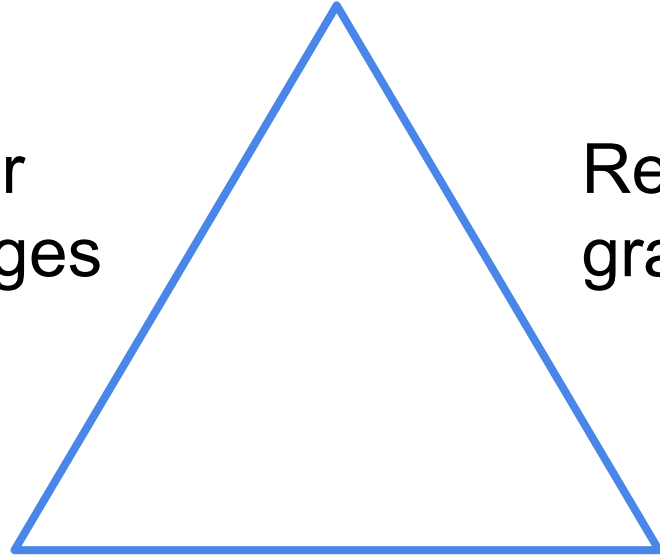
Grammars



Machines

Regular
languages

Regular
grammars



Finite state
automata

baa!

baaa!

baaaa!

baaaaa!

...

baa!
baaa!
baaaa!
baaaaa!
...

$S \rightarrow bT$
 $T \rightarrow aU$
 $U \rightarrow aV$
 $V \rightarrow aV \mid !$

baa!
baaa!
baaaa!
baaaaa!
...

$S \rightarrow bT$
 $T \rightarrow aU$
 $U \rightarrow aV$
 $V \rightarrow aV \mid !$

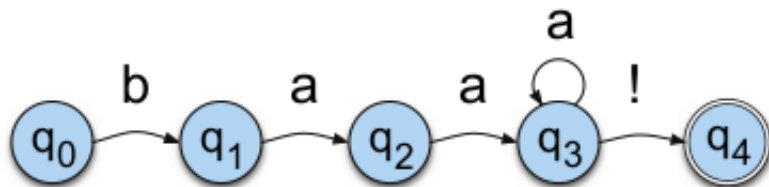
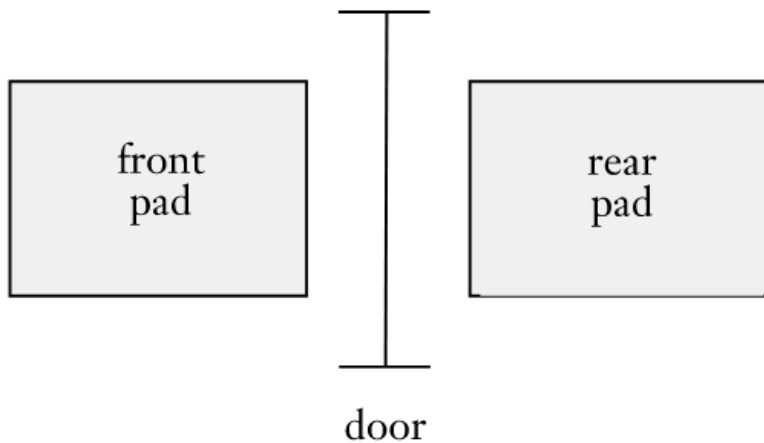
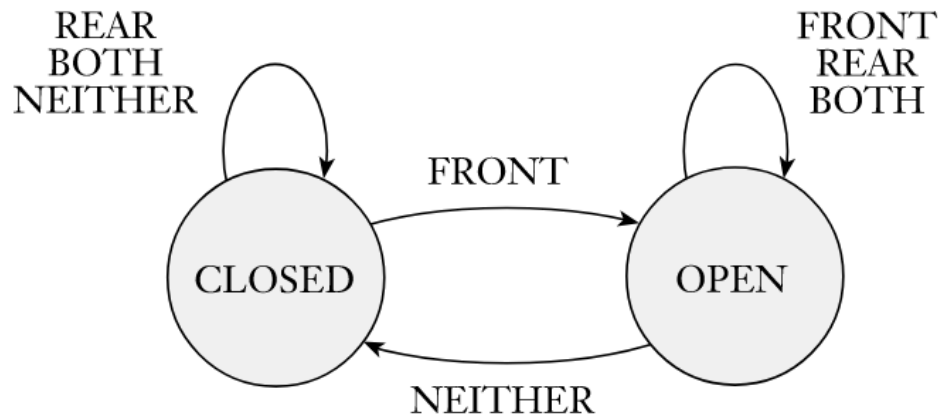
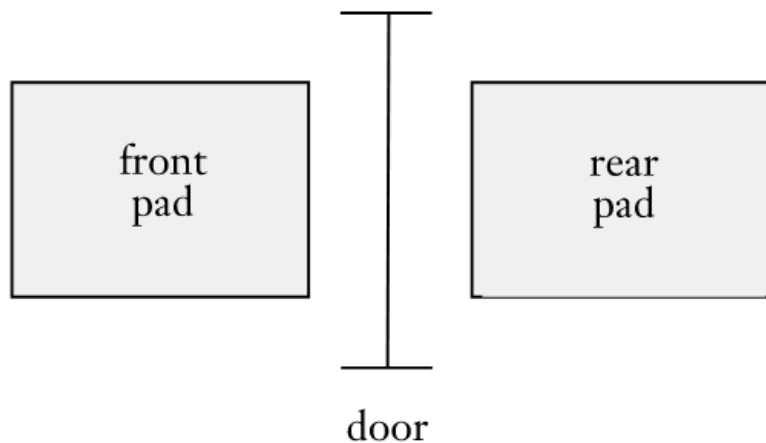


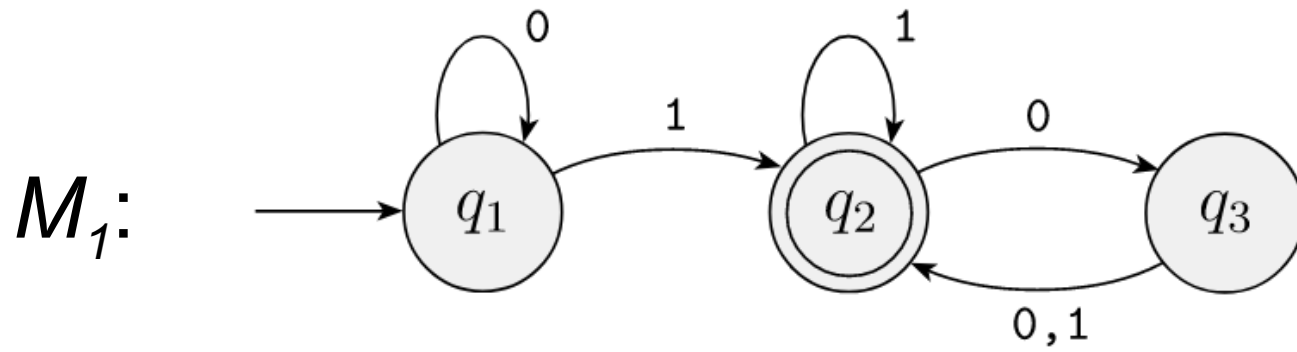
Figure 2.10 A finite-state automaton for talking sheep.

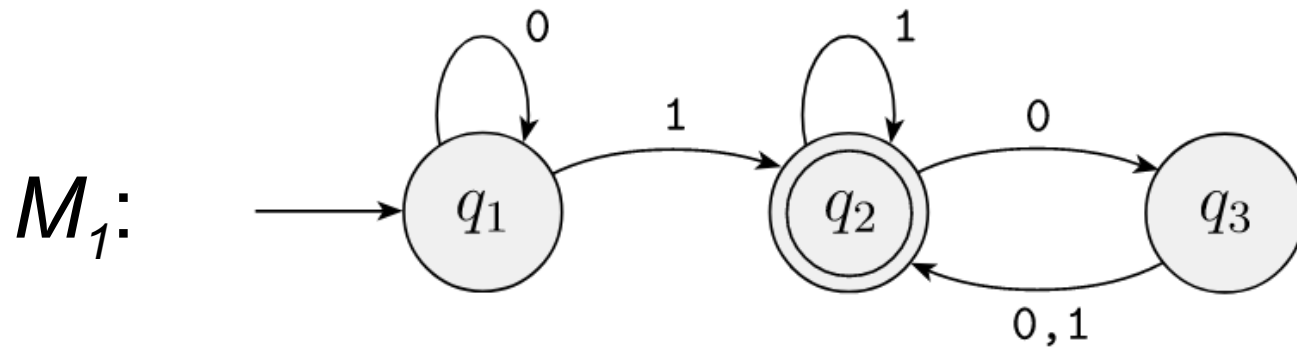
An automatic door



An automatic door



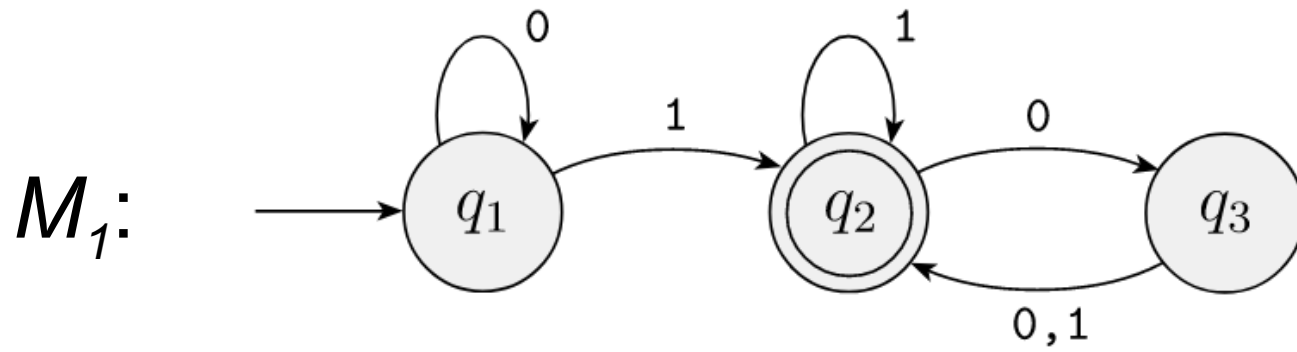




1101

00110

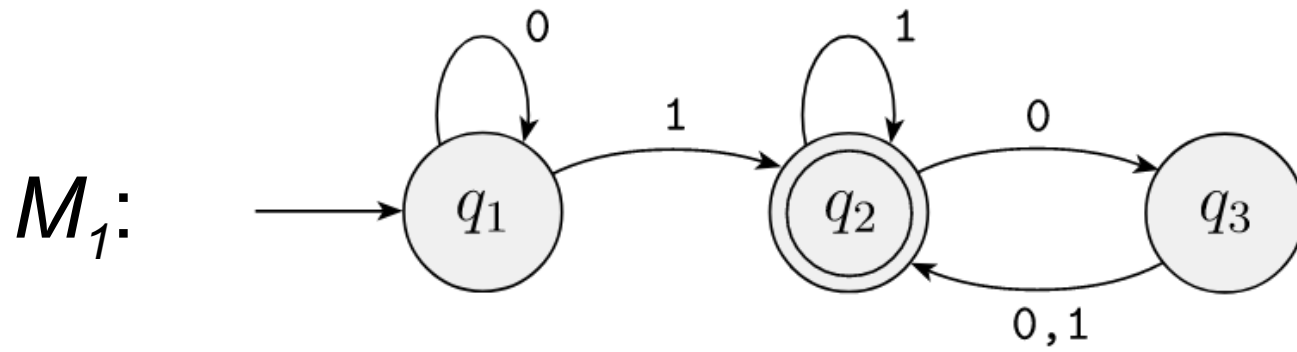
001100



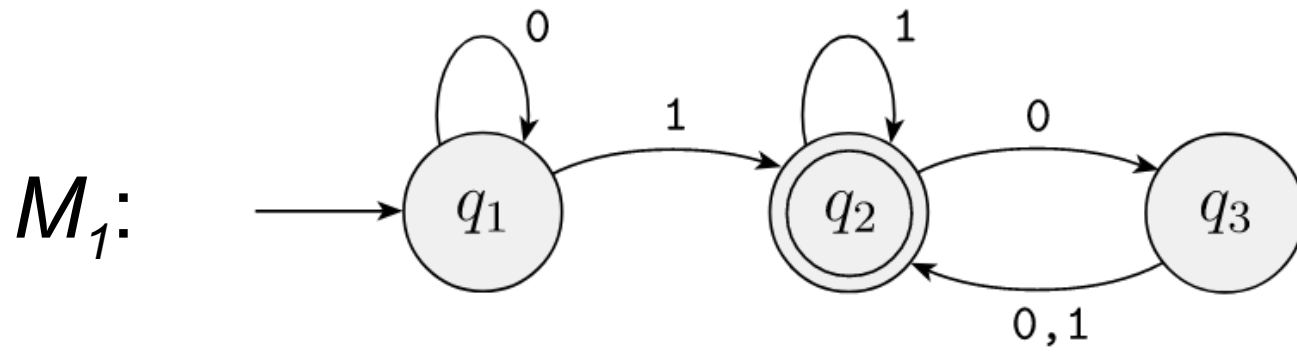
1101

00110

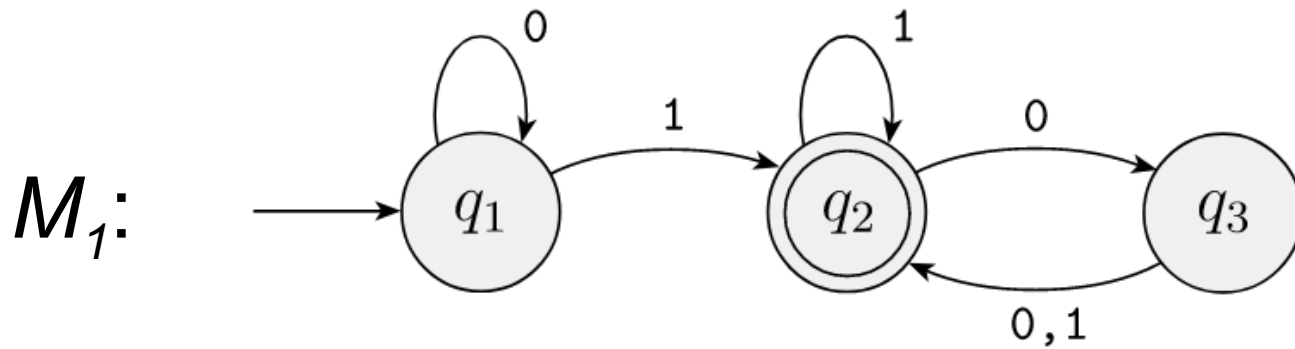
001100



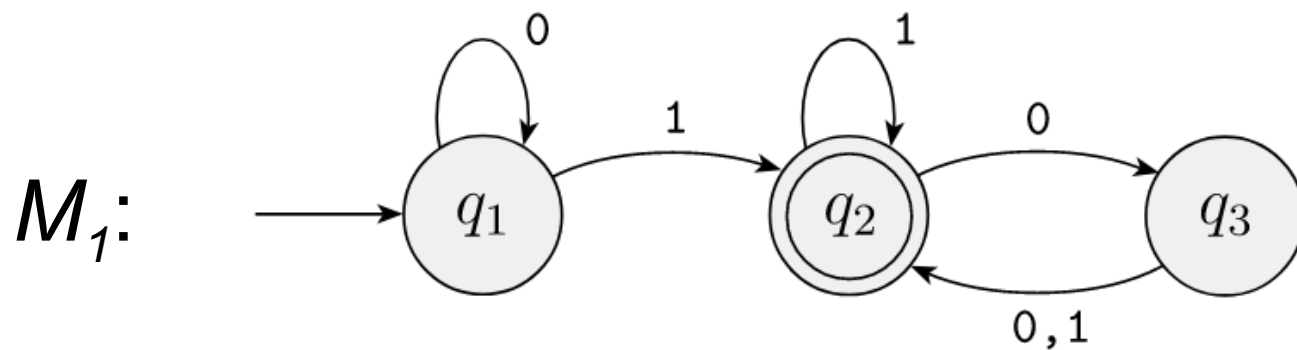
✓ 1101
✗ 00110
001100



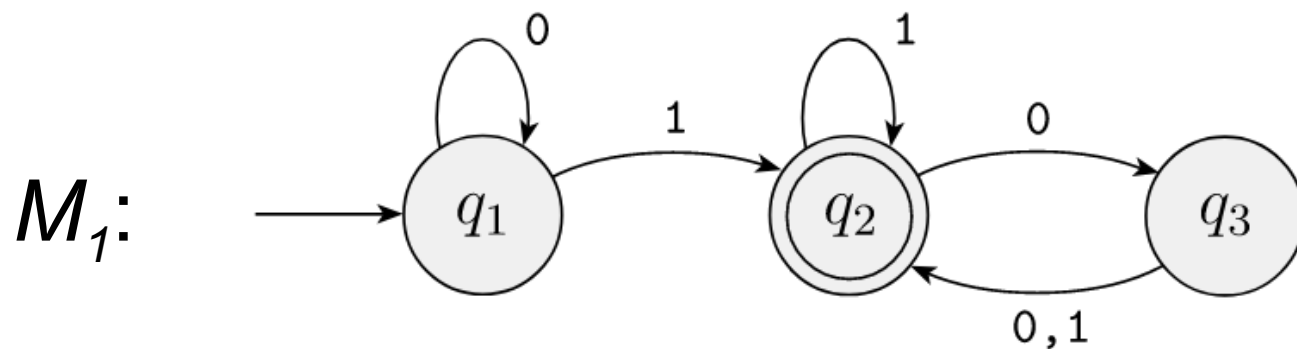
- ✓ 1101
- ✗ 00110
- ✓ 001100



What is the **set of strings** on which this machine will **halt**?



$A = \{w \mid w \text{ contains at least one 1 and}$
an even number of 0s follow the last 1}.



$A = \{w \mid w \text{ contains at least one 1 and}$
an even number of 0s follow the last 1}.

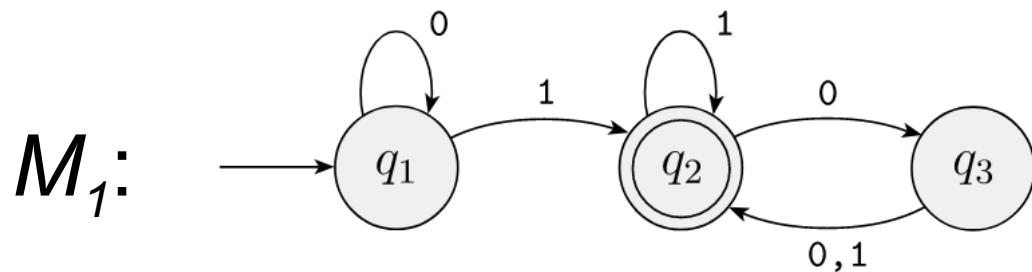
$L(M_1) = A$, or M_1 **recognizes** A ,
or M_1 **accepts** A .

Formal definition

DEFINITION 1.5

A *finite automaton* is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \longrightarrow Q$ is the *transition function*,¹
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.²

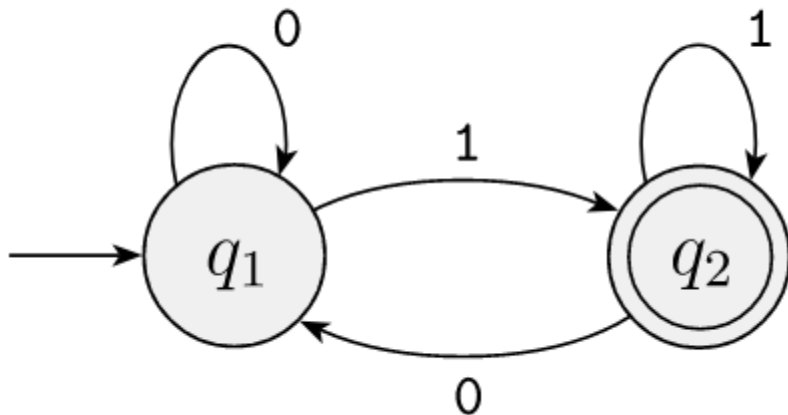


1. $Q = \{q_1, q_2, q_3\}$,
2. $\Sigma = \{0,1\}$,
3. δ is described as

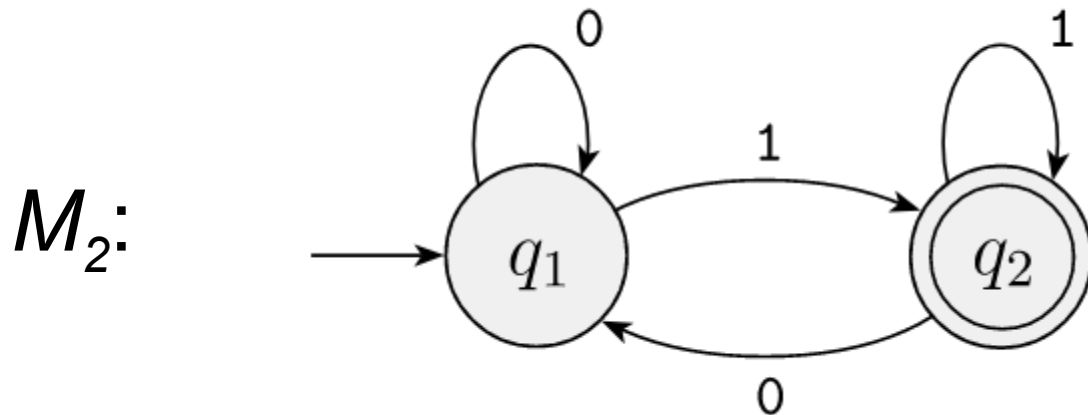
	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2 ,

4. q_1 is the start state, and
5. $F = \{q_2\}$.

M_2 :



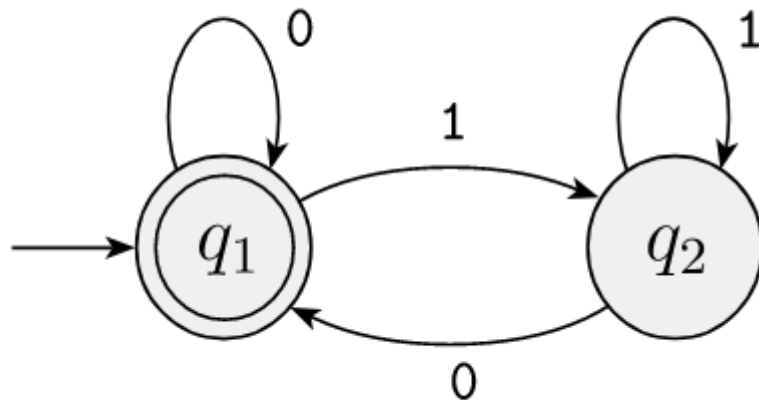
What language does M_2 accept?



What language does M_2 accept?

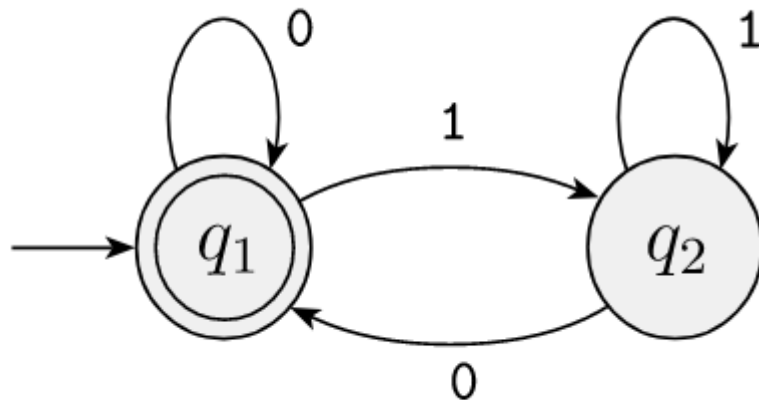
$$L(M_2) = \{w \mid w \text{ ends in a } 1\}.$$

M_3 :



What language does M_3 accept?

M_3 :

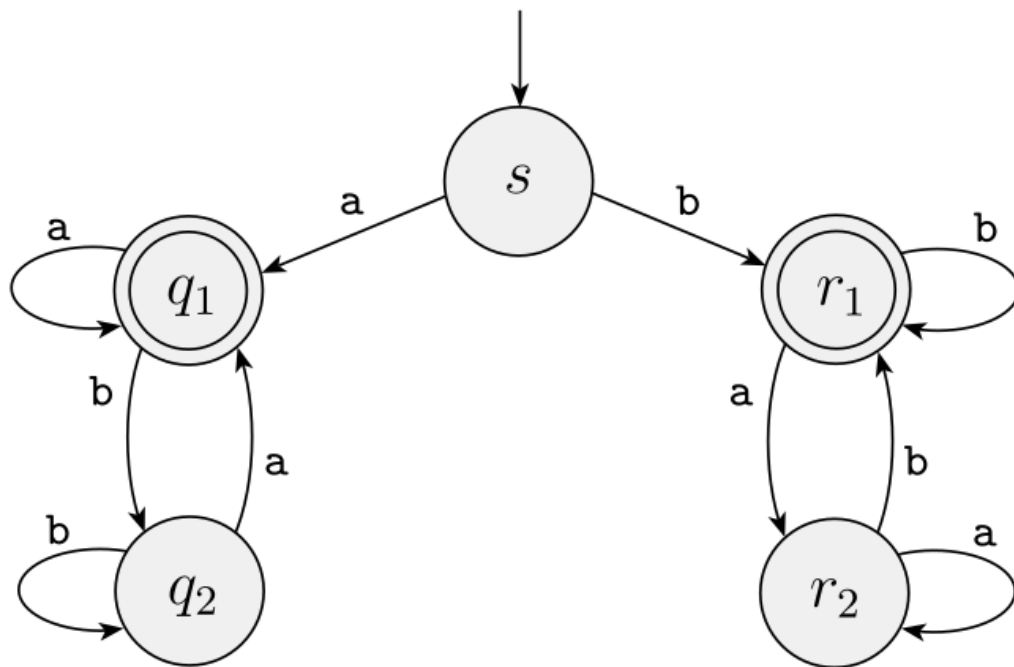


What language does M_3 accept?

$$L(M_3) = \{w \mid w \text{ is the empty string } \varepsilon \text{ or ends in a } 0\}.$$

Sipser, 2013

What language does this machine accept?

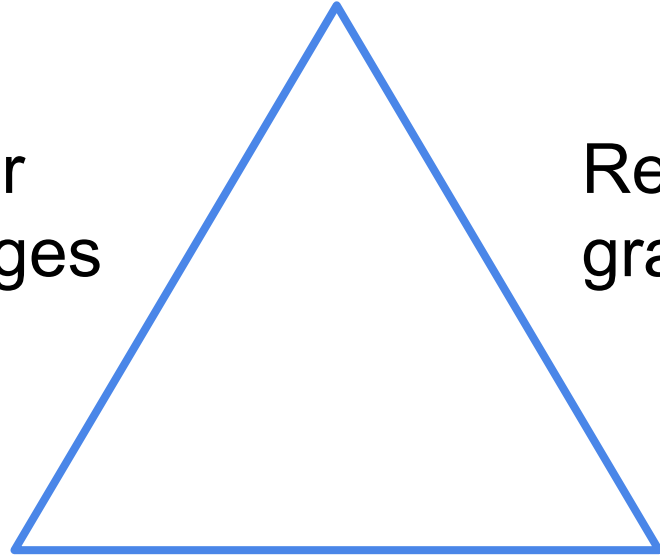


Sipser, 2013

What **comments** or **questions**
do you have?

Regular
languages

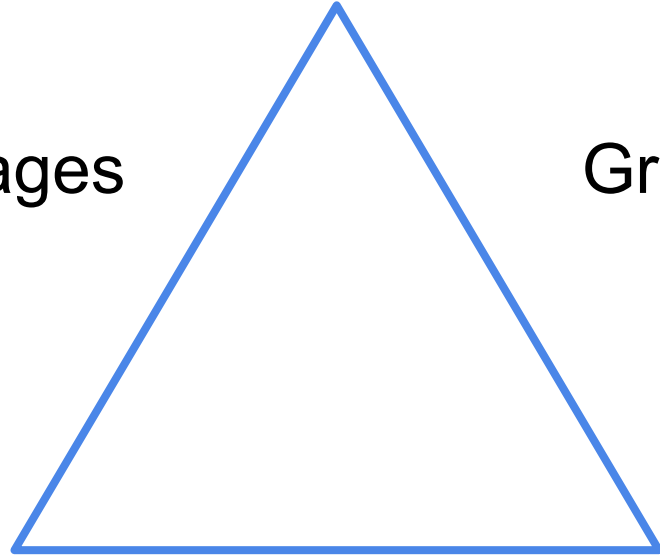
Regular
grammars



Finite state
automata

Languages

Grammars



Machines

The Chomsky hierarchy

- A hierarchy of languages, grammars, and machines.

The Chomsky hierarchy

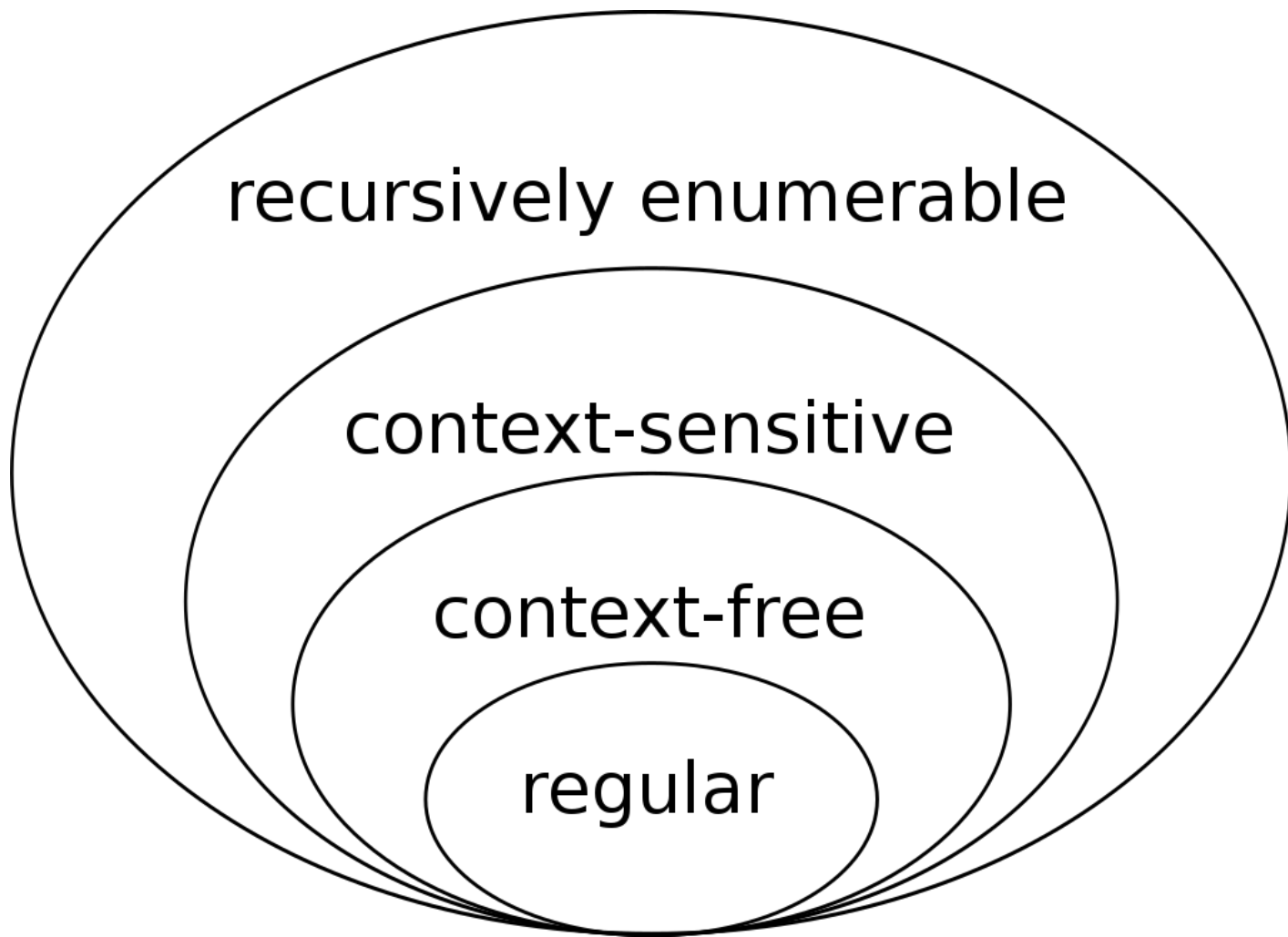
- A hierarchy of languages, grammars, and machines.
- Finite state machines are simple, and near the bottom of the hierarchy.

The Chomsky hierarchy

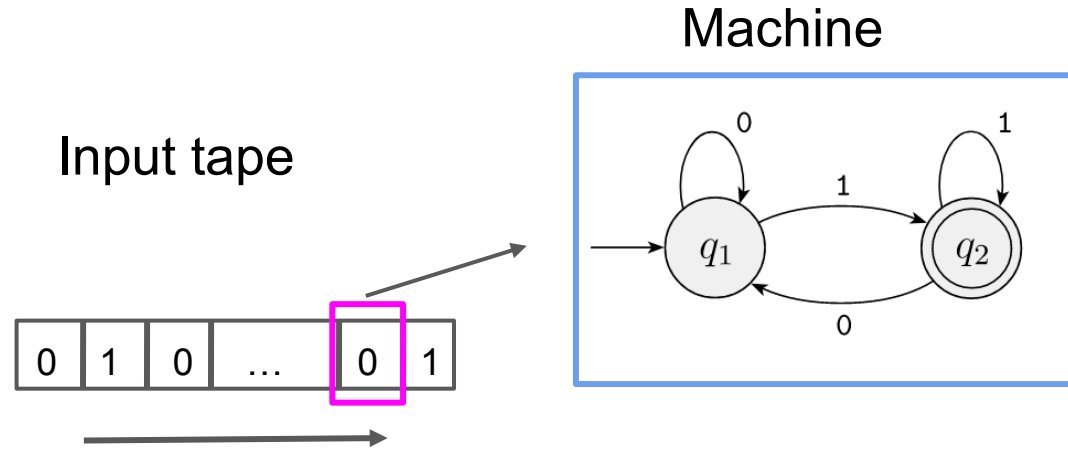
- A hierarchy of languages, grammars, and machines
- Finite state machines are simple, and near the bottom of the hierarchy.
- At the top are Turing machines: mathematical models of computation in general.

The Chomsky hierarchy

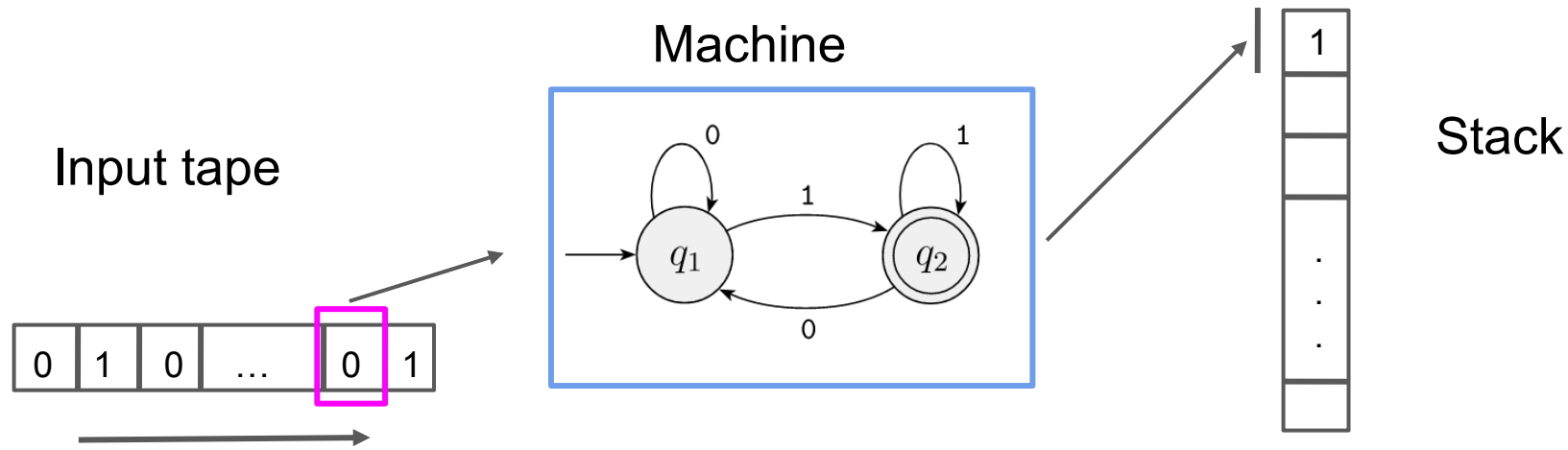
- A hierarchy of languages, grammars, and machines
- Finite state machines are simple, and near the bottom of the hierarchy.
- At the top are Turing machines: mathematical models of computation in general.
- The hierarchy also contains other theoretical objects that will be familiar to many linguists.



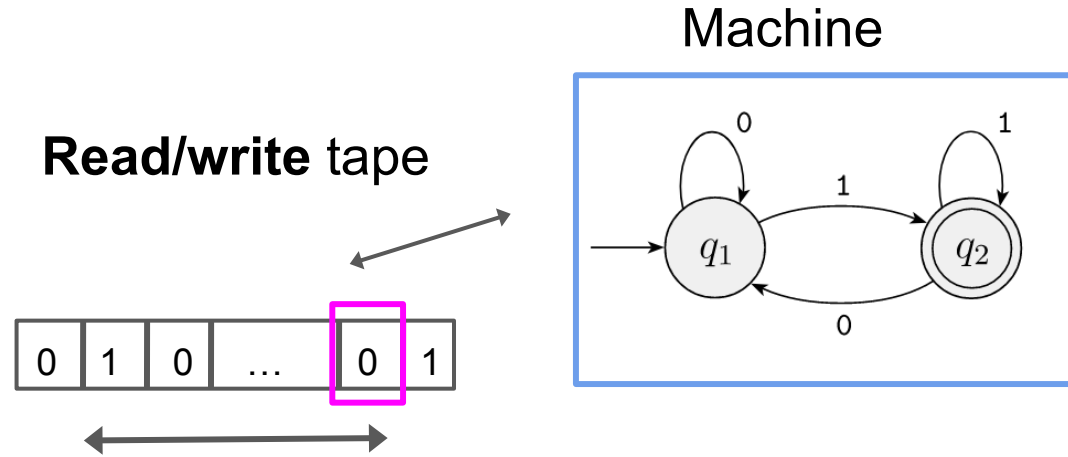
Levels of the hierarchy are distinguished in large part by the type of **external memory** available to the corresponding machine.



- A **finite state automaton** (corresponding to **regular languages**) reads input sequentially from an input tape, and has **no** external memory - all memory is in its states.



- A **pushdown automaton** (corresponding to **context-free languages**) reads input sequentially from an input tape, and has a **stack** as external memory - can access only the **top** item.



- A **Turing machine** reads **and writes** to the same infinite tape. This simple device is the standard abstract model of **computation in general**.

Grammar	Languages	Automaton	Production rules (constraints)*	Examples ^[3]
Type-0	Recursively enumerable	Turing machine	$\alpha A \beta \rightarrow \delta$ (no constraints)	$L = \{w w \text{ describes a terminating Turing machine}\}$
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$	$L = \{a^n b^n c^n n > 0\}$
Type-2	Context-free	Non-deterministic pushdown automaton	$A \rightarrow \alpha$	$L = \{a^n b^n n > 0\}$
Type-3	Regular	Finite state automaton	$A \rightarrow a$ and $A \rightarrow aB$	$L = \{a^n n \geq 0\}$

* Meaning of symbols:

- a = terminal
- A, B = non-terminal
- $\alpha, \beta, \gamma, \delta$ = string of terminals and/or non-terminals
 - α, β, δ = maybe empty
 - γ = never empty

Turing machines and the limits of computation

Hilbert's *decision problem*

Or “*Entscheidungsproblem*”:



David Hilbert
1862-1943

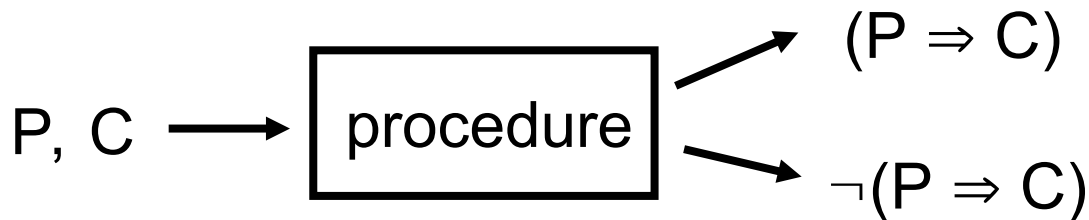
Hilbert's *decision problem*



David Hilbert
1862-1943

Or “*Entscheidungsproblem*”:

Want a procedure that accepts premises P and a conclusion C , and *determines* whether C follows from P .

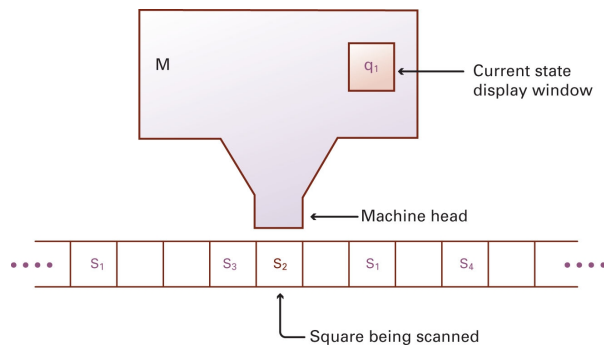


But it was not to be

Turing showed that **no such procedure exists.**



Alan Turing
1912 –1954

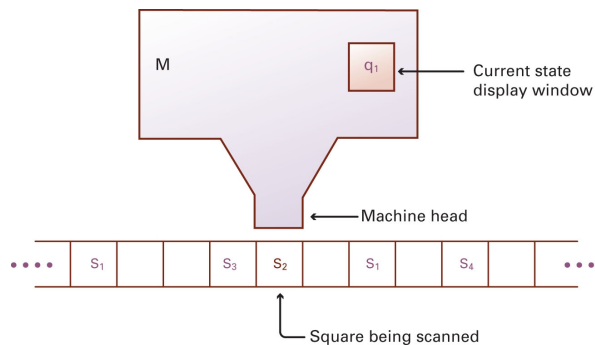


But it was not to be

Turing showed that **no such procedure exists.**



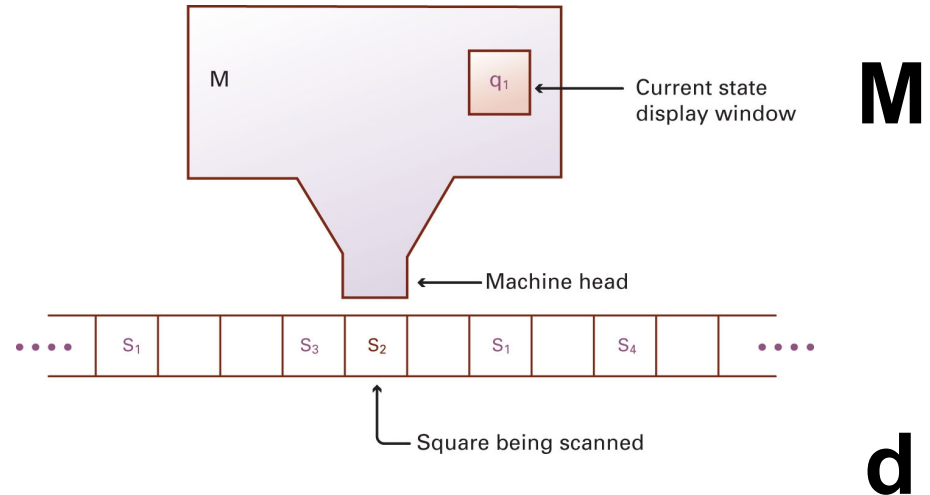
Alan Turing
1912 – 1954



There exist claims that cannot be decided computationally.

The halting problem

Given a Turing machine **M**, with input data **d**, will **M** ever **halt**, or will it **loop endlessly**?



The halting problem

Will this program ever halt?

```
print "Hello, world!"
```

The halting problem

Will this program ever halt?

```
print "Hello, world!"
```



The halting problem

Will this program ever halt?

```
i = 1
while i < 5:
    print "Hello, world!"
```


The halting problem

Will this program ever halt?

```
i = 1
while i < 5:
    print "Hello, world!"
```



The halting problem

Will this program ever halt?

```
i = 1
while i < 5:
    i = new_value(i)
    print "Hello, world!"
```

The halting problem

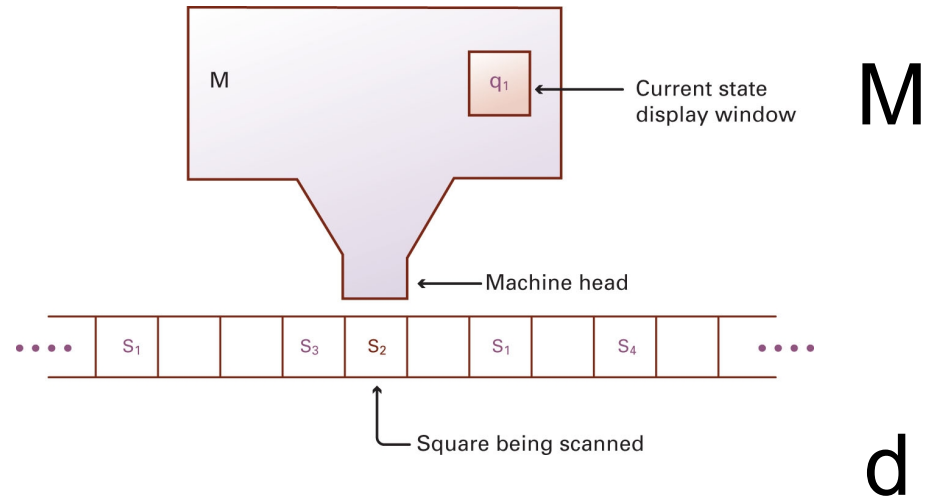
Will this program ever halt?

```
i = 1
while i < 5:
    i = new_value(i)
    print "Hello, world!"
```

?

The halting problem

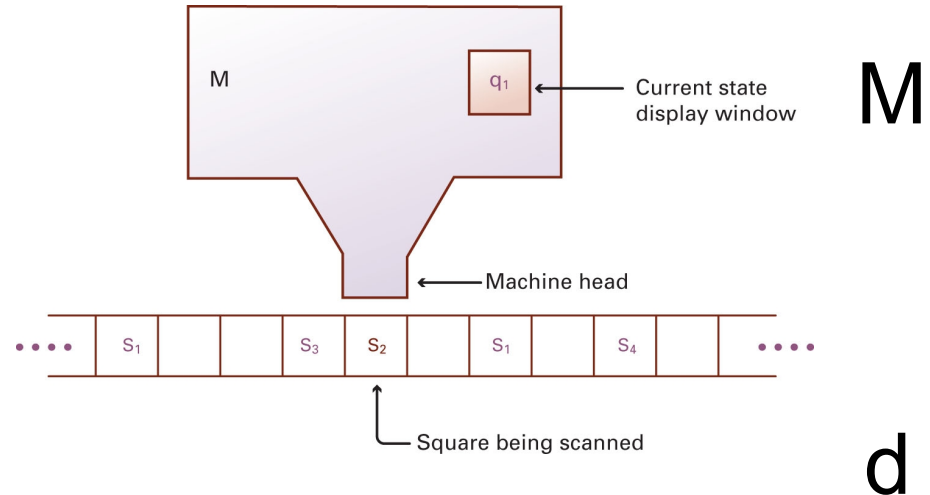
Given a Turing machine M , with input data d , will M ever halt, or will it loop endlessly?



The halting problem

Given a Turing machine M , with input data d , will M ever halt, or will it loop endlessly?

This question cannot be decided computationally.

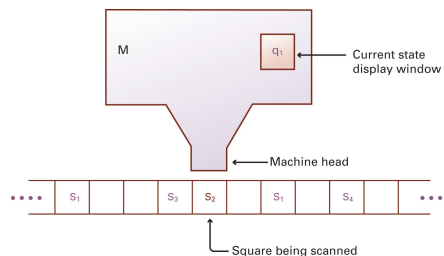


Warmly recommended



Geoff Pullum

Scooping the loop snooper



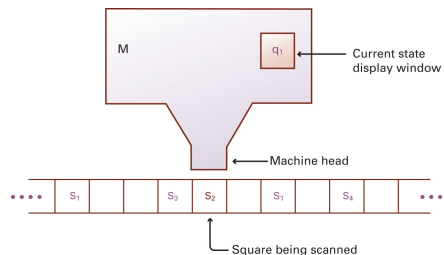
Warmly recommended



Geoff Pullum

Scooping the loop snooper

an elementary proof of
the undecidability of the
halting problem.

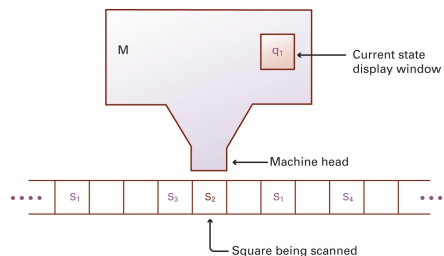


Warmly recommended



Scooping the loop snooper

an elementary proof of
the undecidability of the
halting problem.





No program can say what another will do.

Now, I won't just assert that, I'll prove it to you:
I will prove that although you might work till you drop,
you can't predict whether a program will stop.

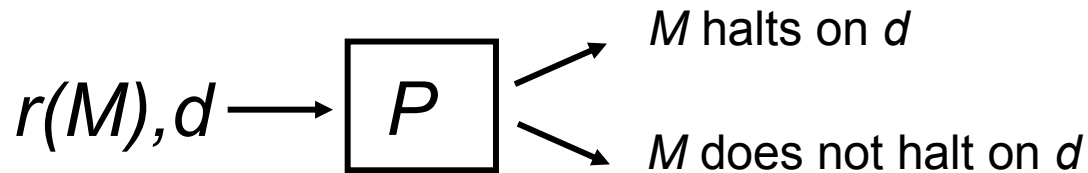
Imagine we have a procedure called P
that will snoop in the source code of programs to see
there aren't infinite loops that go round and around;
and P prints the word "Fine!" if no looping is found.

You feed in your code, and the input it needs,
and then P takes them both and it studies and reads
and computes whether things will all end as they should
(as opposed to going loopy the way that they could).

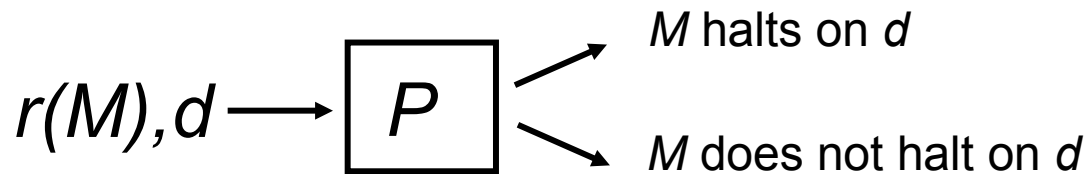
Well, the truth is that P cannot possibly be,
because if you wrote it and gave it to me,
I could use it to set up a logical bind
that would shatter your reason and scramble your mind.

...

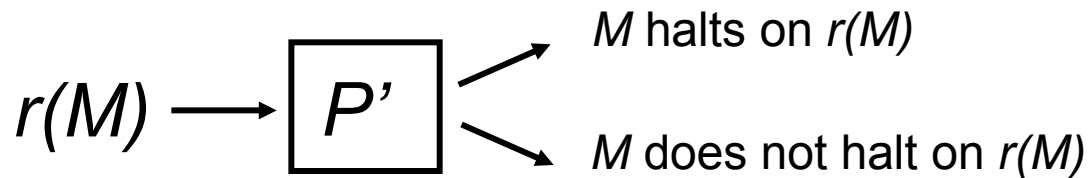
Assume: $\exists$ a procedure P that determines whether a machine M will halt on data d .



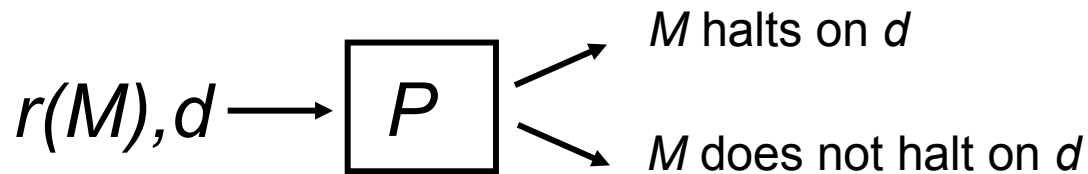
Assume: $\exists$ a procedure P that determines whether a machine M will halt on data d .



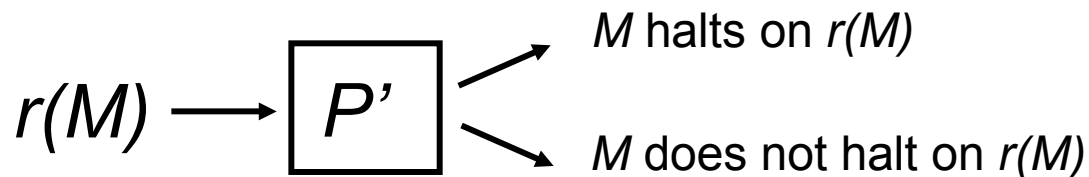
Then P' must also exist:



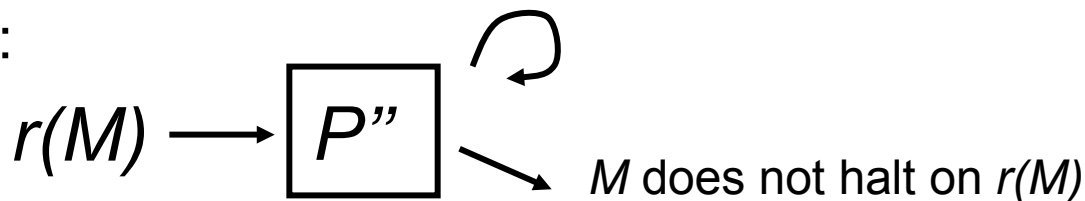
Assume: $\exists$ a procedure P that determines whether a machine M will halt on data d .



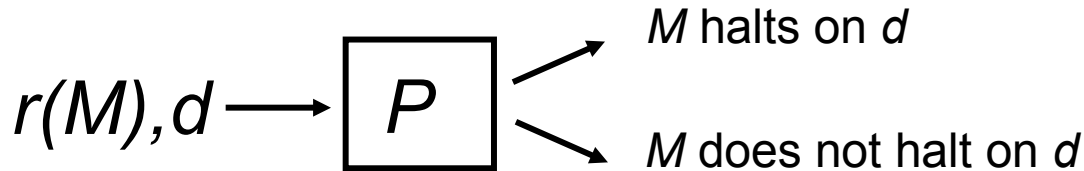
Then P' must also exist:



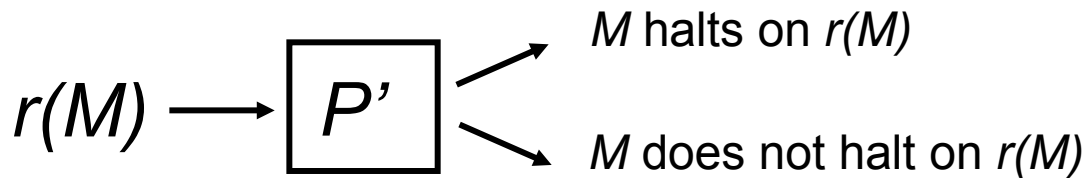
And so must P'' :



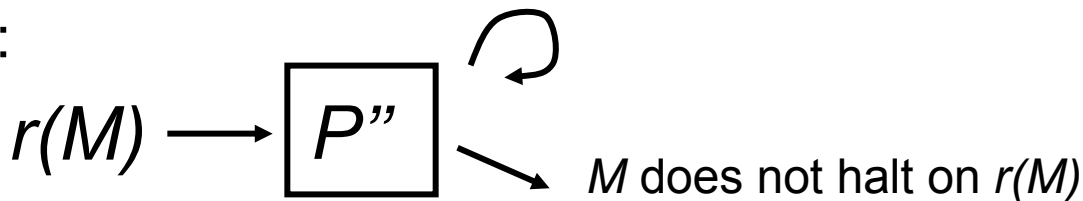
Assume: $\exists$ a procedure P that determines whether a machine M will halt on data d .



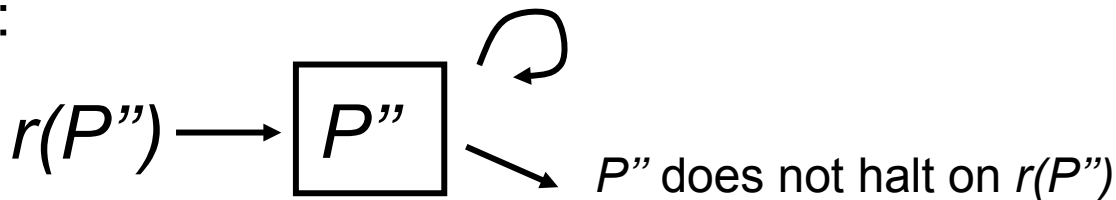
Then P' must also exist:



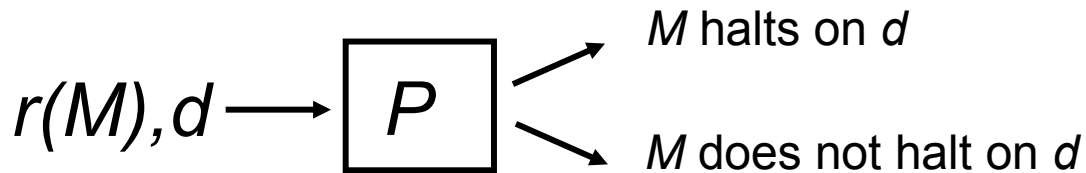
And so must P'' :



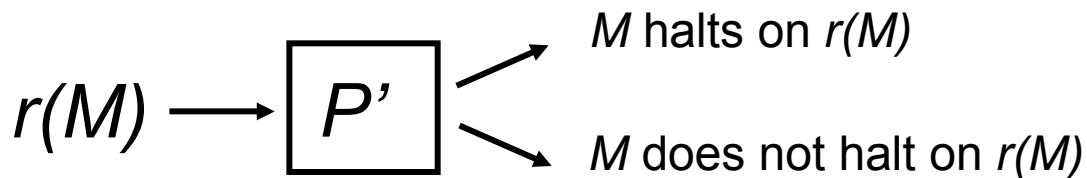
Run P'' on itself:



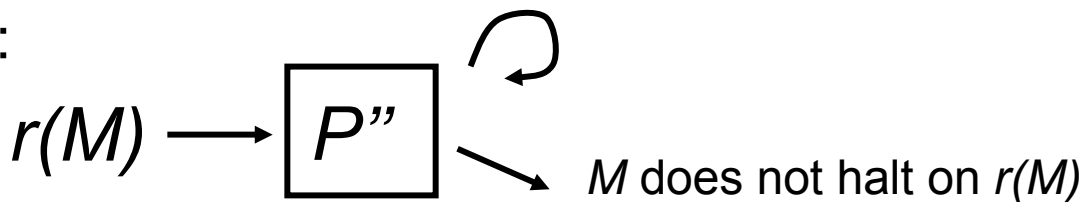
Assume: $\exists$ a procedure P that determines whether a machine M will halt on data d .



Then P' must also exist:

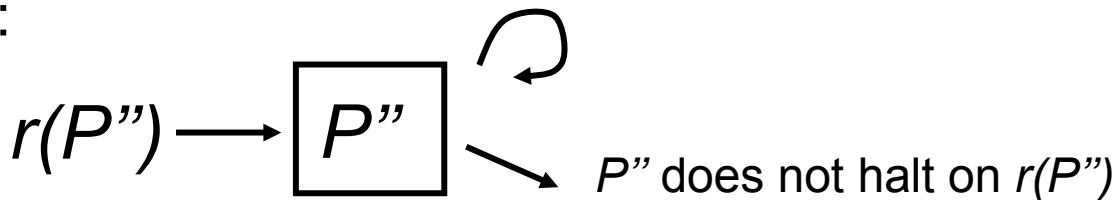


And so must P'' :

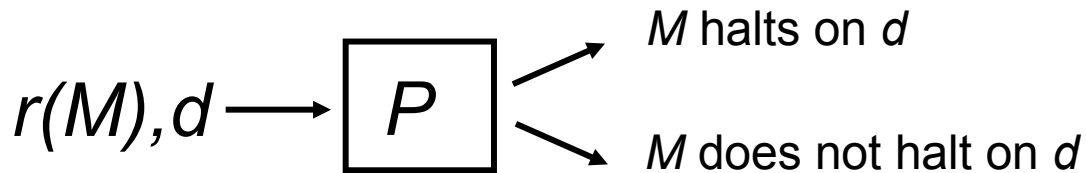


Run P'' on itself:

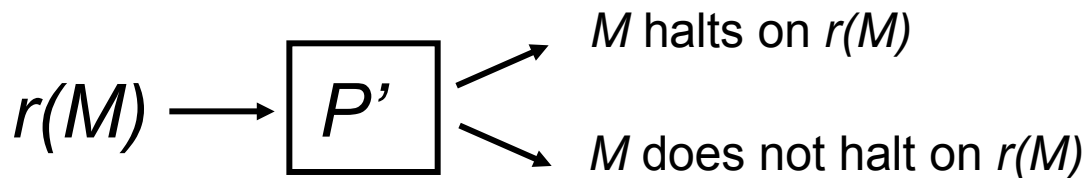
X



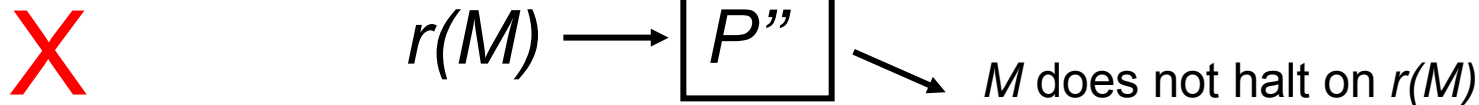
Assume: $\exists$ a procedure P that determines whether a machine M will halt on data d .



Then P' must also exist:



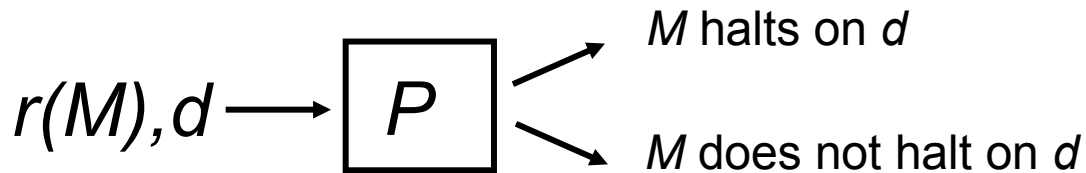
And so must P'' :



Run P'' on itself:

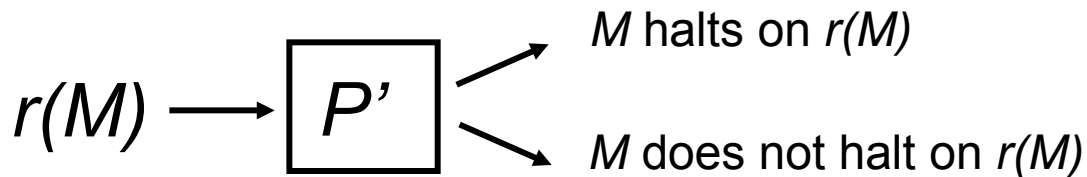


Assume: $\exists$ a procedure P that determines whether a machine M will halt on data d .



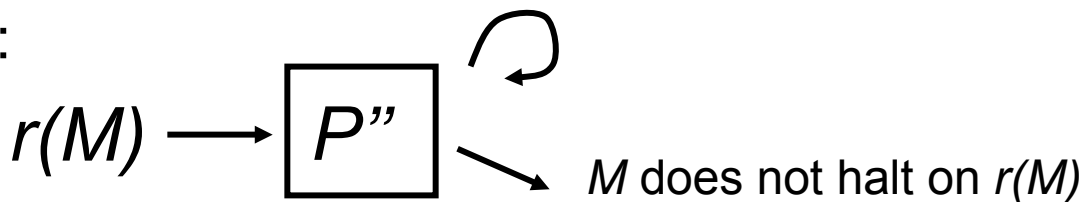
Then P' must also exist:

X



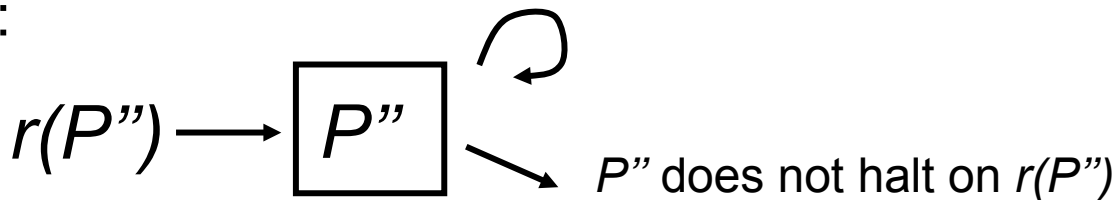
And so must P'' :

X



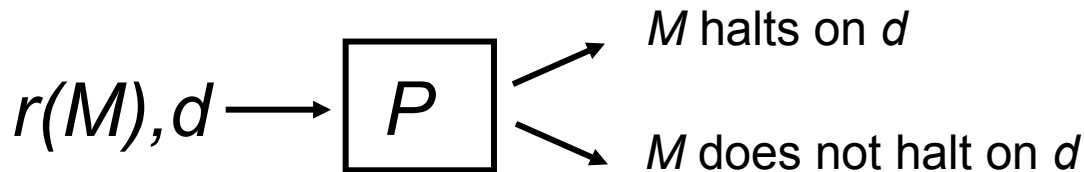
Run P'' on itself:

X



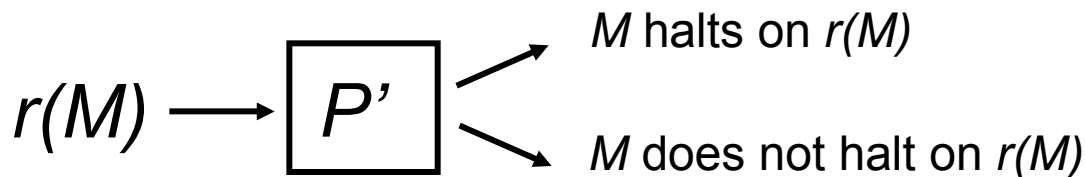
Assume: $\exists$ a procedure P that determines whether a machine M will halt on data d .

X



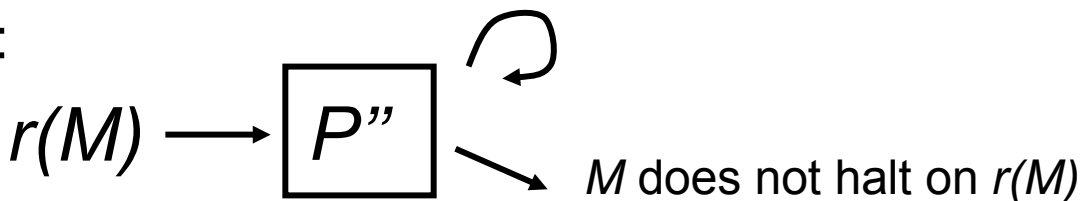
Then P' must also exist:

X



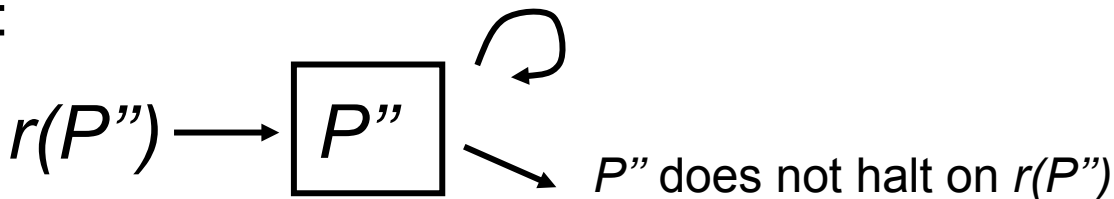
And so must P'' :

X



Run P'' on itself:

X



The main point

There exist claims that cannot be decided computationally.

Wait! I'm a linguist! What does this have to do with human language?

Relevance to human language

- [Chomsky 1956](#) showed that the syntax of human languages is not merely regular (finite state), but at least context-free.

Relevance to human language

- [Chomsky 1956](#) showed that the syntax of human languages is not merely regular (finite state), but at least context-free.
- But [Swiss German syntax](#) is demonstrably not just context-free - at least **mildly context-sensitive**.

Relevance to human language

- [Chomsky 1956](#) showed that the syntax of human languages is not merely regular (finite state), but at least context-free.
- But [Swiss German syntax](#) is demonstrably not just context-free - at least **mildly context-sensitive**.
- Recent work [has treated](#) different levels of language in terms of the **subregular** hierarchy - requiring machines **simpler** than finite-state automata.

Example: SPE-style rules in phonology

Epenthesize [ɪ] between stridents:

$$\emptyset \rightarrow \text{ɪ} \quad / \quad [+strident] \quad ___ \quad [+strident]$$

E.g. in English:

- wɪfz → wɪfɪz
- dʒʌdʒz → dʒʌdʒɪz
- dagz → dagz

Example: SPE-style rules in phonology

Devoice word-final obstruents:

$$[-\text{sonorant}] \rightarrow [-\text{voice}] \quad / \quad ___ \#$$

E.g. in German:

- rad → rat
- rat → rat

Example: SPE-style rules in phonology

General form:

$$A \rightarrow B \quad / \quad C _ D$$

Example: SPE-style rules in phonology

General form:

$$A \rightarrow B \quad / \quad C __ D$$

Simple finite-state **transducer**:



Phonology and the Chomsky hierarchy

- Regular mappings (equivalently finite state transducers) are an **upper bound** on SPE-style rules, and more generally on phonology.

Phonology and the Chomsky hierarchy

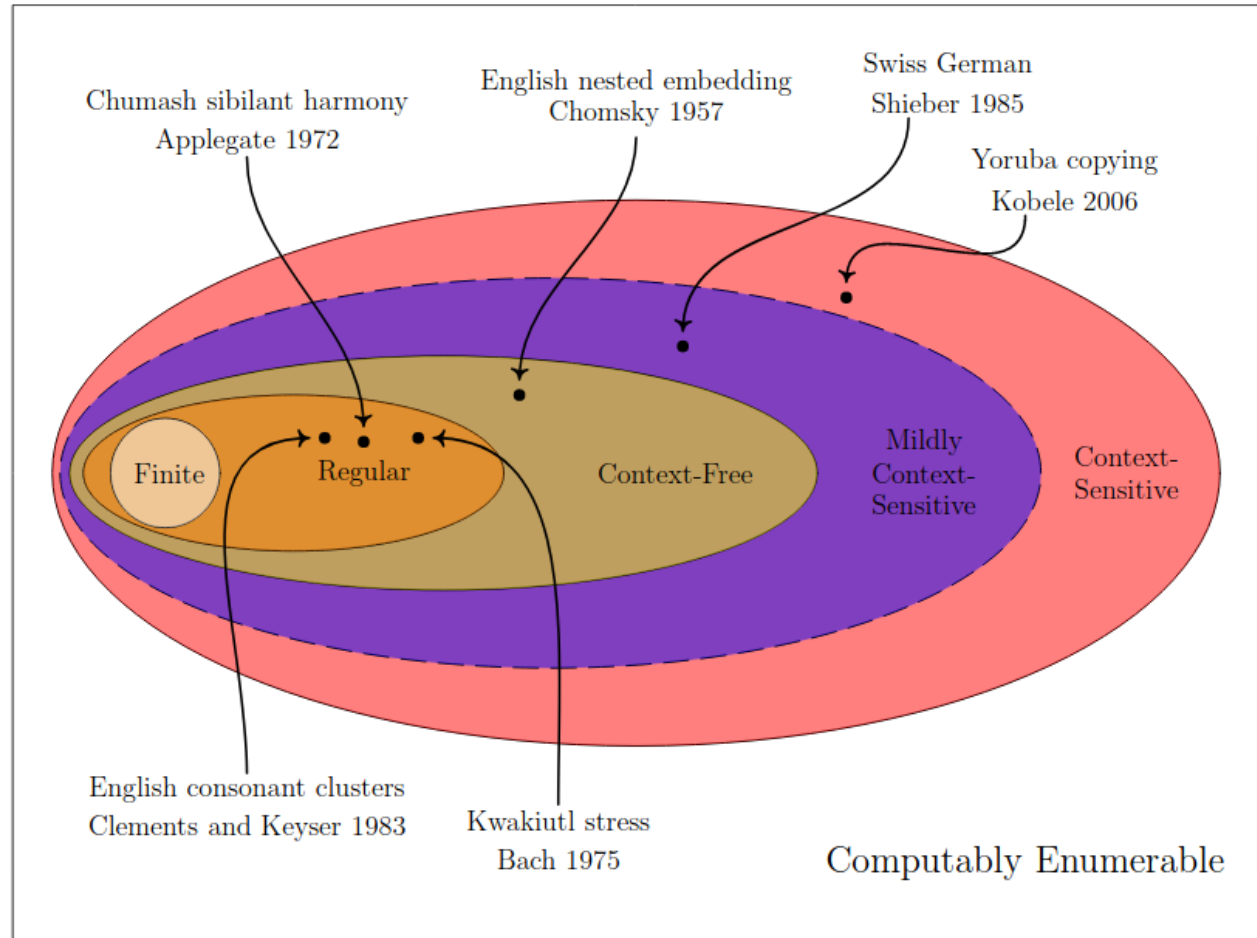
- Regular mappings (equivalently finite state transducers) are an **upper bound** on SPE-style rules, and more generally on phonology.
- But this is **too loose** a characterization.

Phonology and the Chomsky hierarchy

- Regular mappings (equivalently finite state transducers) are an **upper bound** on SPE-style rules, and more generally on phonology.
- But this is **too loose** a characterization.
- Claim: phonology is **subregular**.

Human language and the Chomsky hierarchy.

Heinz 2016



The large message

Ideas that linguists care about can be located within an organizing theoretical structure, named for a linguist, that is central to computer science, and that includes, at its head, the standard abstract model of computation.



Linguistics and
computation
share **deep roots**.

Enough thinking for now - let's start doing.

On to regular expressions in Python

```
>>> p = re.compile(r'\b(\w+)\s+\1\b')
>>> p.search('Paris in the the spring').group()
'the the'
```